

#Unit. 2

#Software Design -

- Process of transforming requirements into a structured solution that can be implemented in programming languages
- It defines architecture, components, interfaces & data flows needed to build the software system
- Objectives:
 - (i) Convert SRS into design models
 - (ii) Improve quality, maintainability, scalability
 - (iii) Reduce complexity
 - (iv) Enable reusability
 - (v) Provide guidelines for developers

~~#From Informal to formal design~~ - Importance:

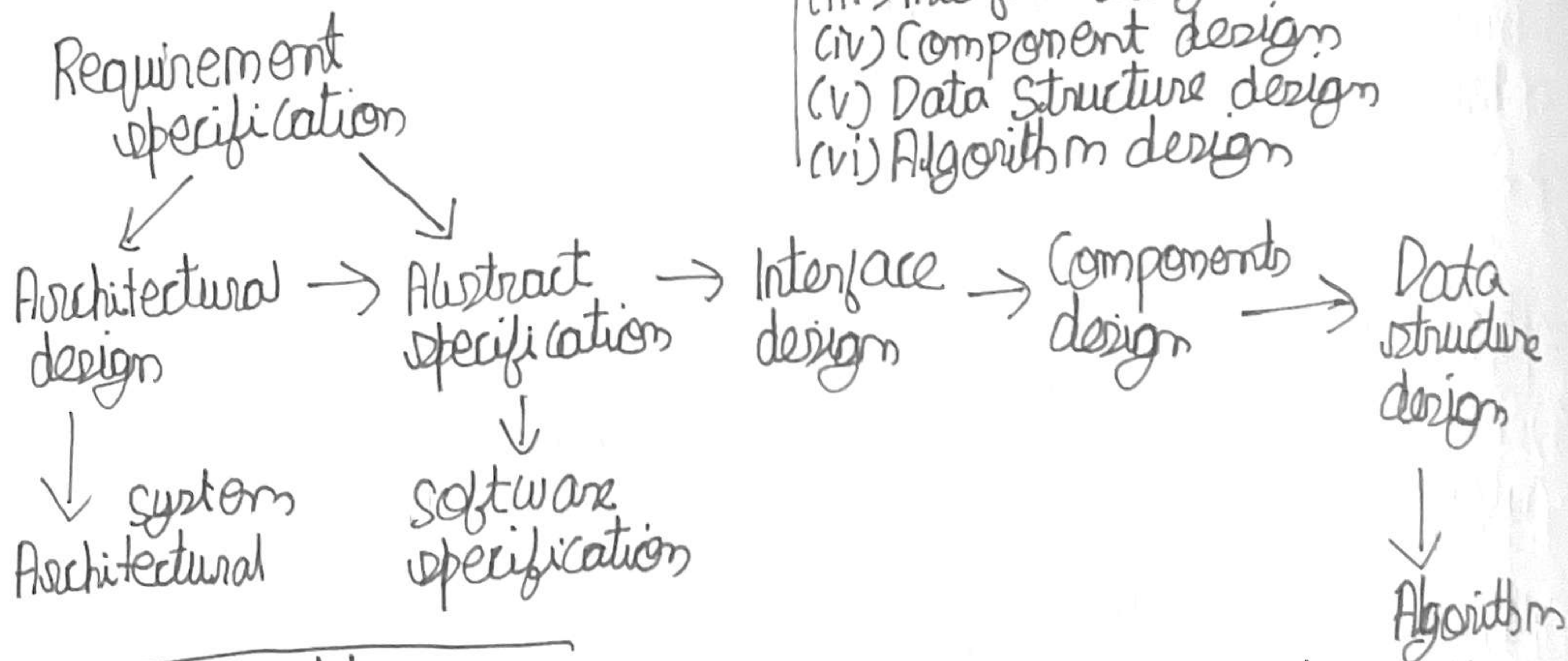
~~Informal design outline~~ - Characteristics:

- (i) Correctness
- (ii) Completeness
- (iii) Efficiency
- (iv) Modularity
- (v) Maintainability
- (vi) Scalability

- Design Phases:

- (i) Architectural design
- (ii) Abstract specification
- (iii) Interface design
- (iv) Component design
- (v) Data Structure design
- (vi) Algorithm design

#Phases in the design process -



#Design description -

- (i) Graphical notations: used to display component relationships
- (ii) Program description: based on programming languages but with more flexibility to represent abstract concepts
- (iii) Informal text: Natural language description

#Software Design Concepts -

- (i) Abstraction
- (ii) Cohesion
- (iii) Modularity
- (iv) Coupling
- (v) Design Patterns
- (vi) Encapsulation
- (vii) Separation of concerns
- (viii) Architectural design
- (ix) Information hiding
- (x) Refinement

~~# Method dependencies -~~

Software Design Classification -

(i) Based on level of design

- High Level Design (HLD)
- Low Level Design (LLD)

(ii) Based on Design approach

- Structured design
- Object Oriented Design (OOD)

(iii) Based on architectural style

- Architectural design
- Interface design
- Data design
- Component level design

Process of Software Design -

1) High Level Design (HLD):

- Overall system architecture
- Modules & subsystems
- Tech stack & database design
- External interfaces

2) Low Level Design (LLD):

- Classes & functions
- Data structures & algorithms
- Function logic & pseudocode

Design Strategies -

1) Functional Design:

- Functional viewpoint
- Shared system state

2) Object Oriented Design:

- Interacting objects
- Manages its own state

Mixed Strategy Design:

- Combines functional & object oriented approaches
- Designers chose the best approach for each subsystem

Design Quality:

- Depends on system requirements & priorities
- Should be efficient, reliable & maintainable
- Apply to both functional & object oriented designs

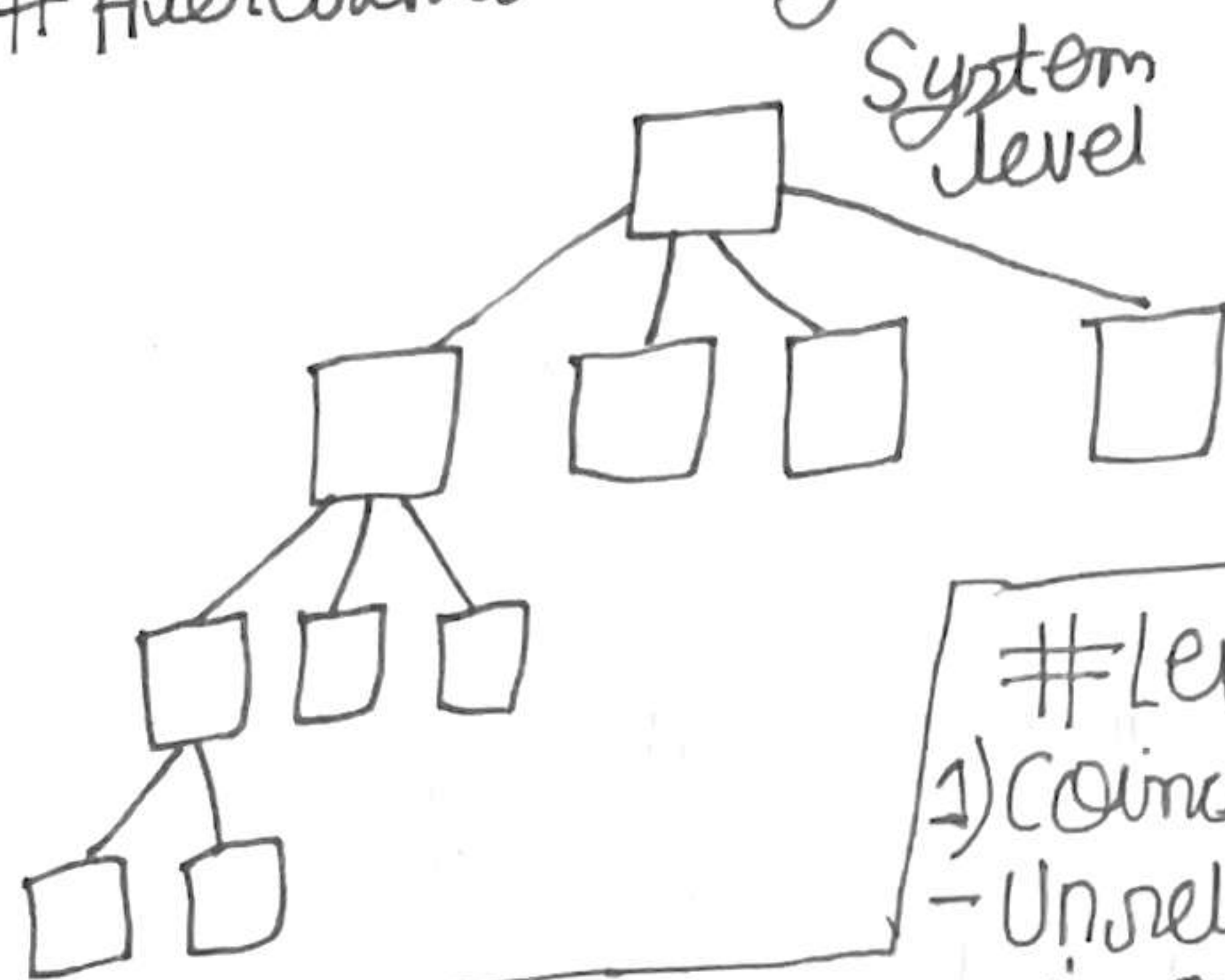
Cohesion -

- Measures how well the elements of a module being together
- Module should perform one single logical function
- High cohesion is desirable because changes remain localized

Cohesion as a design attribute -

- Cohesion may sometimes be difficult to classify precisely
 - Inheritance can weaken cohesion
 - To evaluate cohesion, examine both the class & its superclasses
 - Class browser/tools help analyze cohesion
- 

Hierarchical design structure -



Levels of Cohesion (Weak → Strong):

1) Coincidental Cohesion (Weakest)
- Unrelated tasks are grouped randomly in one module

~~Top down design~~ 2) Logical Cohesion (Weak):
- Elements performing similar kinds of operations are grouped

3) Temporal Cohesion (Weak):
- Tasks executed at the same time are grouped

4) Procedural Cohesion (Weak-Medium):
- Elements follow a specific control sequence

5) Communicational Cohesion (Medium):
- Elements operate on the same data set

~~6) So~~

~~Method components~~ -

6) Sequential Cohesion (Medium-Strong):
- Output of one component becomes inputs of another

~~A structure~~

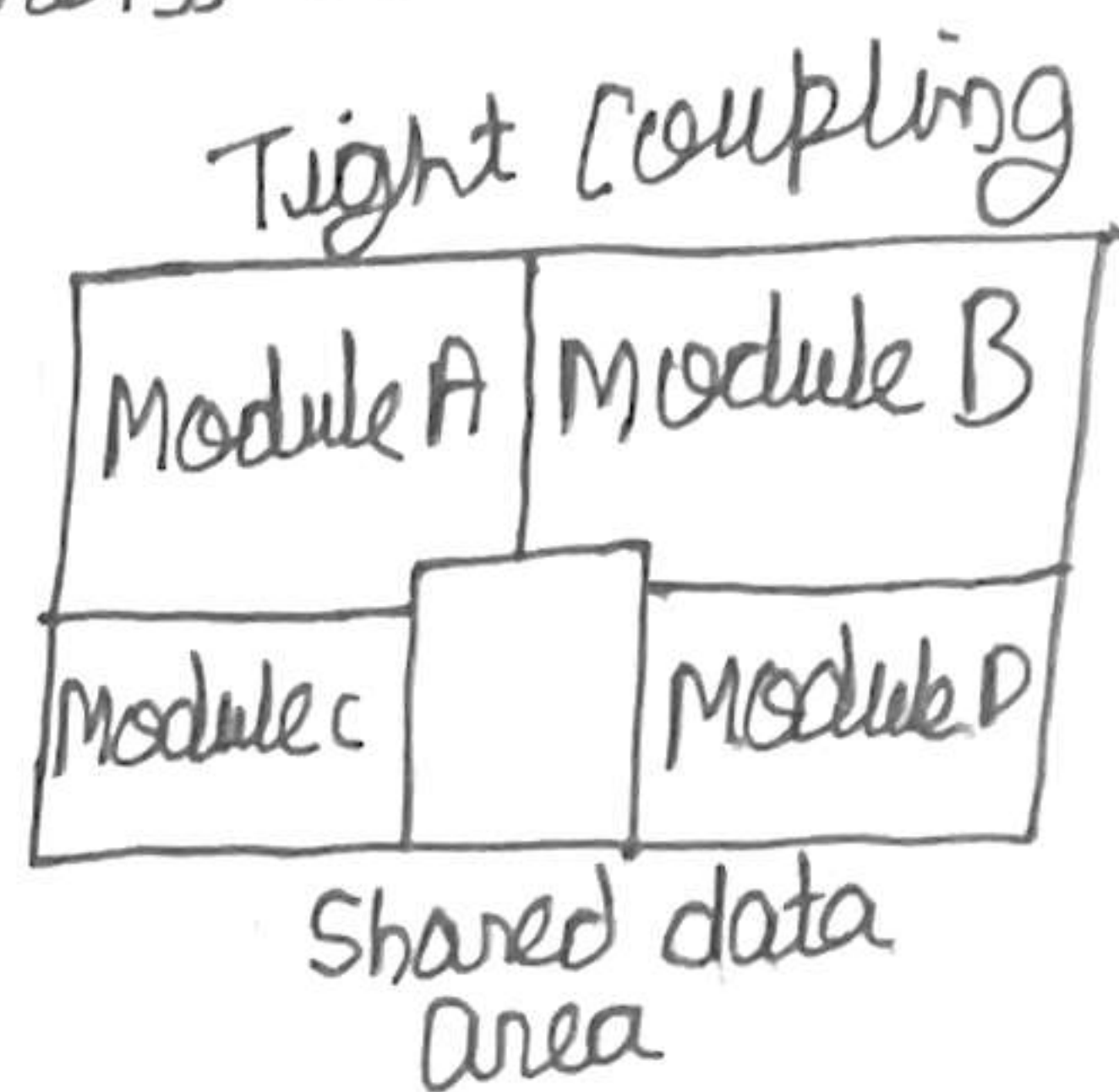
7) Functional Cohesion (Strong):
- Module performs one well defined functions

~~Many methods~~
~~support~~
~~comparable views~~
~~of a system~~

8) Object Cohesion (Strong):
- Operations are related to manipulating object attributes

- ## # Coupling -
- Measures the degree of dependency between modules / components
 - Low (Loose) coupling is desirable because changes in one module do not significantly affect others
 - Tight coupling occurs when modules share data, control information, or modify each other's data / code

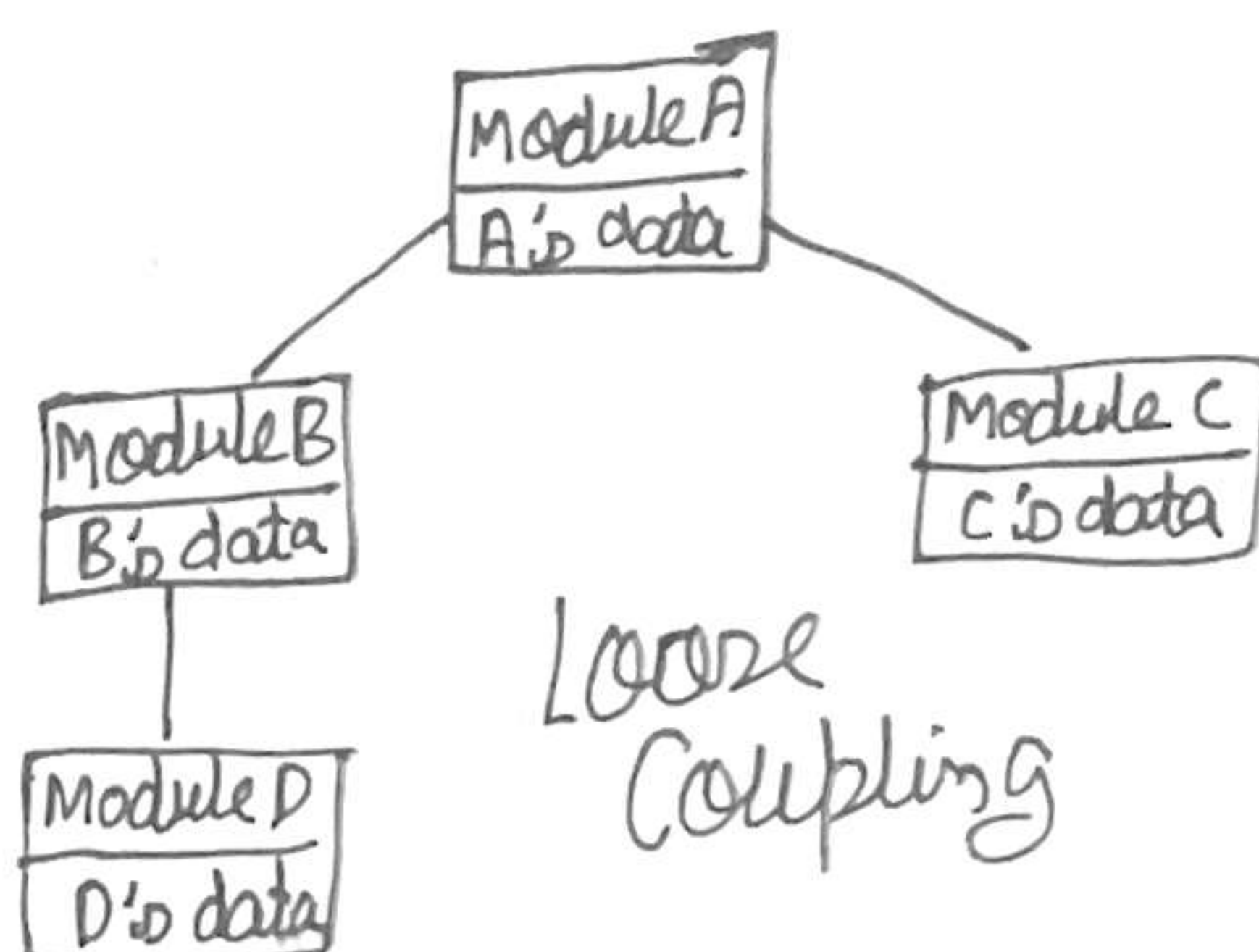
- ## # Causes of Tight Coupling -
- Shared/global variables or data
 - Direct modification of another module's data or code
 - Passing control flags or switches
 - Use of pointers or references



Types / Levels of Coupling -

(Strong → Weak)

- 1) Content Coupling (Worst):
 - One module directly modifies another module's code or internal data
- 2) Common Coupling:
 - Modules share global data or variables
- 3) Control Coupling:
 - One module controls another module's behavior
- 4) Data Coupling (Good):
 - Modules interact by passing only necessary data as parameters
- 5) Message / Stream Coupling (Best):
 - Modules communicate through data streams or message passing



Coupling & Inheritance (OOP) -

- Promotes loose coupling via message passing
- Subclasses depend on superclasses, creating inheritance coupling
- Changes in superclass ~~but~~ may affect all subclasses, so inheritance must be managed

Design process characteristics -

- (i) Design must be implemented explicitly
- (ii) Design should be readable & understandable
- (iii) Design should provide complete picture

Design Quality attributes -

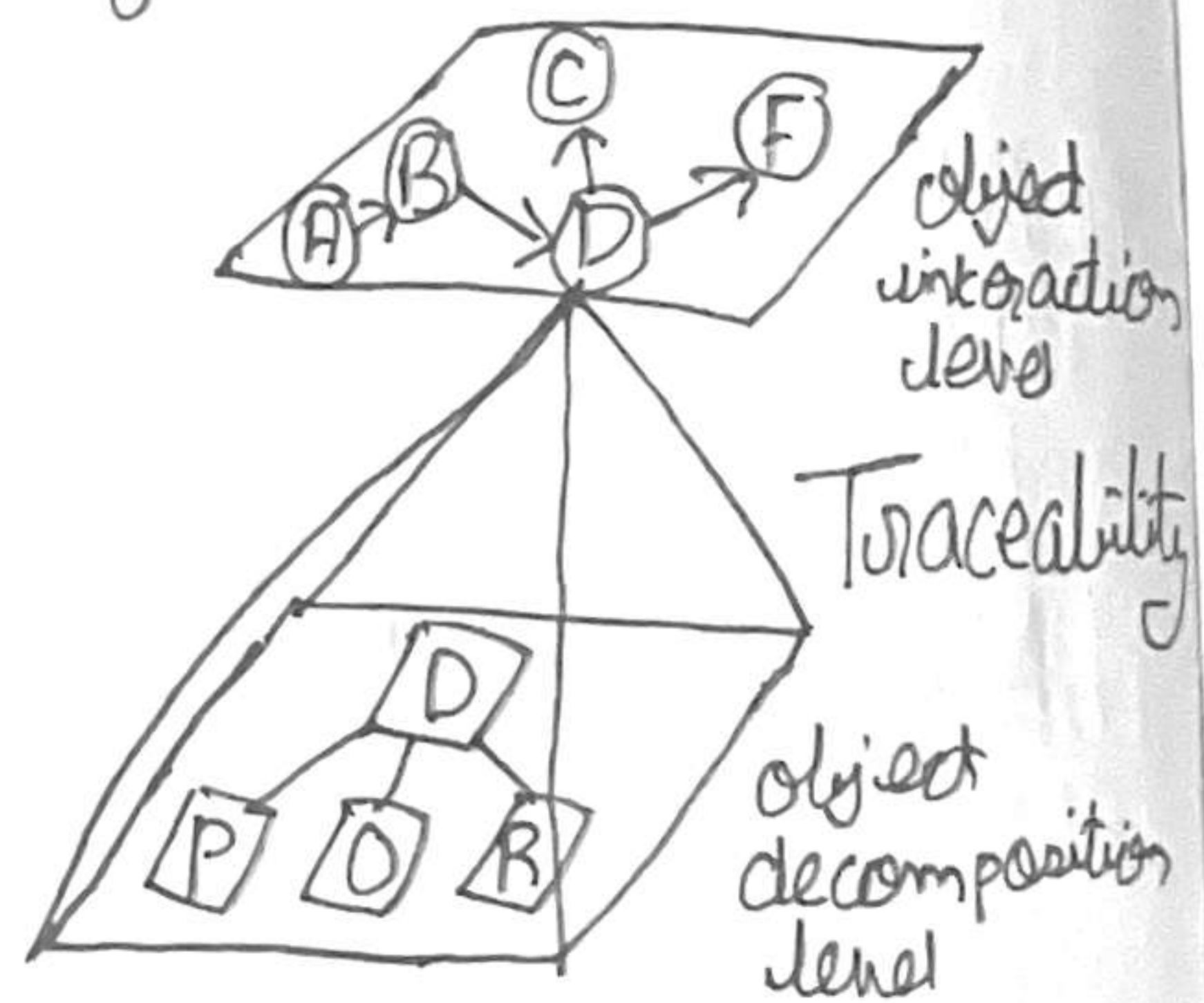
- (i) Functionality
- (ii) Usability (easy to use)
- (iii) Reliability (secure)
- (iv) Performance
- (v) Supportability

Understandability -

- Refers to how easily a software component can be understood
- Influenced by:
 - Cohesion
 - Meaningful names
 - Proper documentation
 - Low complexity
- High complexity & many inter connections reduce understandability

Adaptability -

- Means how easily a system can be modified or extended
- Design is adaptable when:
 - Components are loosely coupled
 - Components are highly cohesive
 - Documentation is clear
 - Relationships are traceable



Adaptability & Inheritance -

- Inheritance improves adaptability
- Deep inheritance can increase complexity
- Systems may require periodic restructuring (refactoring) to maintain adaptability

Abstraction -

- Process of hiding implementation details & showing only essential features of the system

- Purpose:

- Reduce complexity
- Focus on imp aspects
- Improve readability

- Types:

- Data abstraction
- Procedural abstraction
- Control abstraction

eg - ATM machine - user withdraw money without knowing internal banking operations

Modularity -

- Dividing a system into smaller independent modules

- Benefits:

- Easy maintenance
- Reusability
- Product Module

- Example:

Online shopping system modules:

- Login Module
- Payment Module
- Product Module

Each module performs a specific task

Encapsulation -

- Binding data & methods together in a single unit (class) & restricting direct access

- Achieved using: Access modifiers (private, public, protected)

- Benefit:

- Data protection
- Improved security

Information Hiding -

- Internal details of a module are hidden from other modules

- Only necessary interfaces are exposed

- Goal:

- Prevent unintended interference
- Improve system reliability

Separation of Concerns (SoC) -

- Dividing system into distinct sections, each addressing a separate concern

- Benefit

- Improved flexibility
- Easier debugging
- Better scalability

Design Patterns -

- Reusable solutions to common design problems

- Types:

- Creational (Object creation)
- Structural (composition of classes & objects)
- Behavioral

« Refer to 3rd chapter for detail »

- Improve code reusability

Refinement (Stepwise refinement) -

- Breaking a complex problems into smaller manageable parts

- Approach:

- Top down design
- Gradual detailing

Architecture design -

- Defines overall system structure & interaction between components

- eg -

- Layered architecture
- Client Server
- MVC
- Microservices

Interface Design -

Focus:

- User interface
- API design
- Interaction between modules

Refactoring -

- Improves internal code structure without changing external behavior. Enhances maintainability

UML (Unified Modeling Language) <OR> Case Tool Diagram

- Standard visual modeling language

- Used to design, visualize, specify, & document software systems

- Widely used in Object Oriented Analysis and Design (OOAD)

- Why:

- Common visual language for developers
- Understand system structure and behavior
- Improves communication & documentation
- Works with different development methodologies (Agile, Waterfall)

- Diagram Categories:

(a) Structured diagrams (Static view)

- Shows structure of the system

eg:

- Class diagram
- Object diagram
- Component diagram
- Deployment diagram

(b) Behavioral diagrams (Dynamic view)

- Shows the behavior and interactions of the system

eg:

- Use case diagram
- Sequence diagram
- Activity diagram
- ~~Activity diagram~~
- State machine diagram
- Communication diagram
- Interaction Overview diagram

- Key UML Concepts:

(a) Class

- Blueprint of objects
- Contains attributes (data) and ~~attributes~~ methods (operations)

(b) Object

Instance of a class

(c) Relationships (Defines how elements interact)

- Association = basic relationship between classes
- Aggregation = weak whole-part relationship
- Composition = strong whole-part relationship
- Generalization = inheritance relationship
- Dependency = one element depends on another

Use Case Diagram —

- Represents functional requirements from the user's perspective
- Shows actors, use cases, and their relationships

- Purpose:

- Capture functional requirements
- Identify users (actors) and their goals
- Define system boundaries
- Support requirement validation
- Basis for other UML diagrams (Class, Sequence, Activity)

- Key elements:

(a) Actor = external entities interacting with the system

Types:

- Primary = initiates interaction
- Secondary = supports the system
- Can be human users or external systems

- (b) Use Cases
- Represents functions provided by the system
 - eg. Login, Register User, Place Order, Search Books
 - Represents by oval shapes

(c) System Boundary

- Rectangle defining the system scope
- Inside: system functions (use cases)
- Outside: actors

Relationships in Use Case Diagram -

- Association: link between actor & use case
- Include: one use case always includes another
- Extend: optional or conditional behavior
- Generalization: Inheritance between actors or use cases

Steps -

- | | |
|--|---|
| (i) Identify actors | (v) Connect actors to use cases (association) |
| (ii) Identify main use cases | (vi) Add include, extend, generalization relationships |
| (iii) Draw system boundary | (vii) Review diagram to ensure all requirements are covered |
| (iv) Place actors outside & use cases inside | |

Benefits -

- Easy to understand technical & non technical users
- Helps in requirement gathering
- Provides a foundation for system design diagrams
- Useful for test case development

Limitations -

- Does not show internal system logic
- Cannot represent data flow or algorithms
- Provides only a high level view of the system

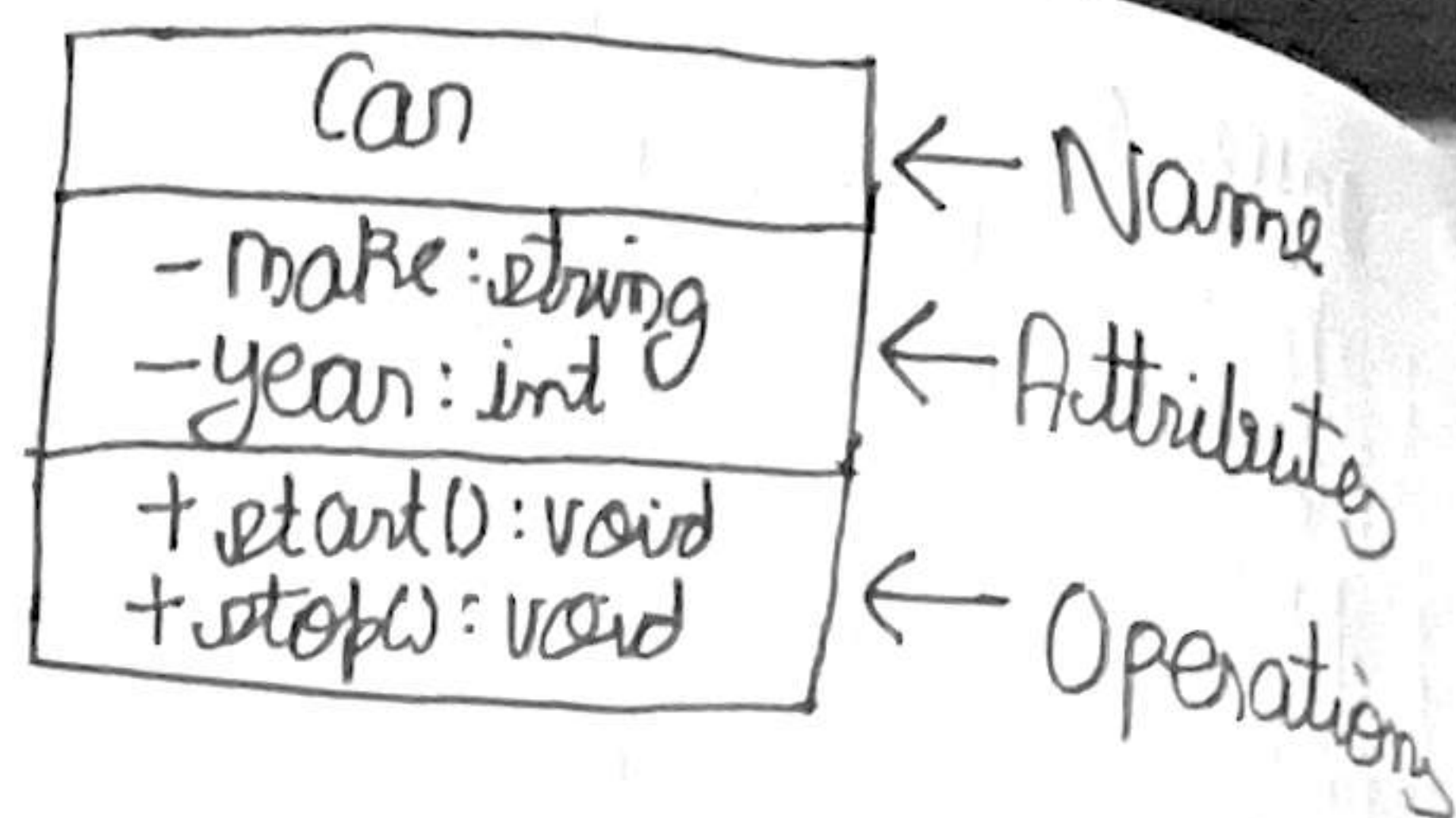
Class Diagrams -

- Most important structural diagrams in UML
- Represent static structure of a system by showing:
 - Classes
 - Attributes
 - Operations (methods)
 - Relationships between classes
 - Used heavily during OOAD

Elements of a class -

- Represented by a rectangle with 3 components:

- Class Name
- Attributes (data)
- Operations (methods)



Relationships in class diagrams -

1) Association:

Basic relationship between classes showing interaction



2) Aggregation (Weak Whole-Part):

Part can exist independently of the whole



3) Composition (Strong Whole-Part):

Part cannot exist without the whole



4) Generalization (Inheritance):

One class inherits properties of another class



5) Dependency:

One class temporarily depends on another



Multiplicity -

- Shows how many instances of one class relate to another

- Common values:

- 1..1 → exactly one
- 0..* → zero or many
- 1..* → at least one

- Eg - Customer → Order = 0..* (a customer can place many orders)

Uses of class diagrams -

- Designing object structure of a system
- Database schema ~~modeling~~ modeling
- Understanding relationships between system components
- Generating code templates

Sequence Diagram -

- Behavioral UML diagram
- Shows how objects interact over time
- Represents the flow of messages between objects in a scenario
- Used to visualize:
 - └ Use case execution
 - └ Method calls
 - └ Object collaboration

Elements of Sequence Diagram -

- (a) Life lines:
 - Vertical dotted lines representing an object's lifespan
- (b) Activation Bar:
 - Rectangles on lifelines showing when an object is alive
- (c) Messages:
 - Arrow showing communication between objects:
 - └ Synchronous: solid arrow with filled head
 - └ Asynchronous: solid arrow with open head
 - └ Return: dashed arrow
- (d) Objects:
 - Placed at the top of the diagram with underlined names

Steps to create a sequence diagram -

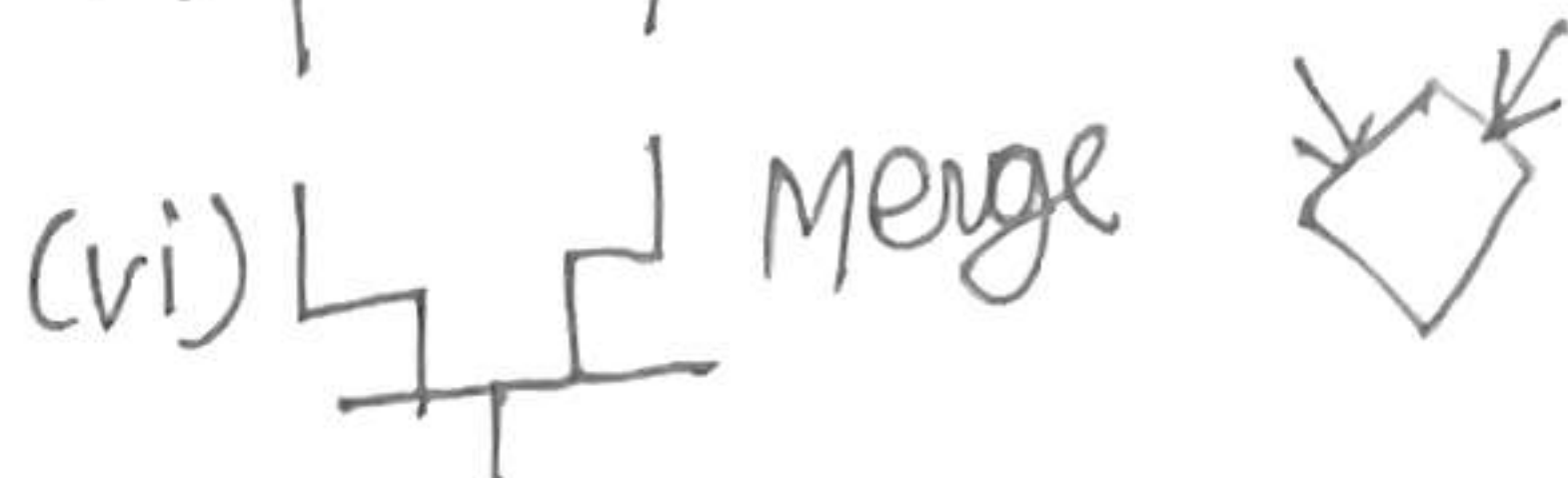
- Identify the scenarios (eg-Login)
- Identify participating objects
- Arrange objects left to right
- Start interaction with action message
- Add message flow & return messages

Activity Diagram -

- Used to model the workflow or business processes of a system
- Shows step by step activities, decisions, parallel tasks, & the flow of control
- Useful for modeling complex logic, use case flows & system operations

Notations:

- (i) Start 
- (ii) Stop 
- (iii) Activity 
- (iv) Decision 



- When to use
 - └ Business processes
 - └ Workflow of a use case
 - └ Logic of complex methods
 - └ Parallel processes in a system

- Steps to draw an activity diagram -
 - (i) Identify all activities of the workflow
 - (ii) Add initial node & final node
 - (iii) Arrange ~~the~~ activities in sequence
 - (iv) Add decisions and merge nodes where conditions exist
 - (v) Use fork and join for parallel tasks
 - (vi) Add swimlanes to distinguish responsibilities
 - (vii) Review and optimize the workflow

- Advantages

- Easy to understand
- Good for communi
- Helps identify automation opportunities
- Detects missing conditions
- Useful for process optimization

- Limitations

- Can become complex for large systems
- Does not show data structures or class relationships
- Hard to maintain if workflows change frequently

Design Patterns

- Reusable solution to a commonly occurring problem ~~in~~ in software design
- They provide templates, not complete code
- Importance
 - Improve software reusability
 - Provides standard solutions
 - Enhance code readability
 - Reduce complexity
 - Make systems easy to agile & modify

- Types -

- (a) Creational
- (b) Structural
- (c) Behavioural

Creational design Patterns

- Focus = object creation mechanisms reduce complexity

- Types -

- Singleton pattern
- Factory method pattern
- Abstract factory pattern
- Builder pattern
- Prototype pattern

Creation al design patterns-

- 1) Singleton Pattern = (i) Ensures only one instance of a class exists
(ii) Used in: Logging, Configuration, Database connections
- 2) Factory Method Pattern = (i) Creates objects without exposing creation logic
(ii) Superclass defines interface; subclasses decide the object
- 3) Abstract Factory Pattern = (i) Provides an interface to
(ii) eg - UI toolkits (Windows/Mac)
- 4) Builder Pattern = (i) Used to construct complex objects step by step
(ii) eg - creating documents, assembling products
- 5) Prototype Pattern = (i) Creates new objects using clones
(ii) Useful when object creation is expensive

Structural Design Patterns -

- Focus = composition of classes and objects

- Types

- Adapter Pattern
- Decorator Pattern
- Facade Pattern
- Composite Pattern
- Proxy Pattern
- Bridge Pattern
- Flyweight Pattern

- 1) Adapter Pattern = (i) Converts interface of one class into another
(ii) Works as a bridge between incompatible interfaces
- 2) Decorator Pattern = (i) Adds new functionality to an object without modifying existing code
(ii) eg - adding features to UI widgets
- 3) Facade Pattern = (i) Provides a simple interface to a complex subsystem
(ii) eg - single API calling multiple modules
- 4) Composite Pattern = (i) Treats individual objects and groups uniformly
(ii) Useful in hierarchical structures (file system)
- 5) Proxy Pattern = (i) Provides a surrogate or placeholder for another object
(ii) eg - virtual proxy, remote proxy, protection proxy
- 6) Bridge Pattern = (i) Separates abstraction from implementation
(ii) Helps extend both independently
- 7) Flyweight Pattern = (i) Minimizes memory usage by sharing common objects
(ii) eg - Game objects (trees)

Behavioural Design Patterns -

- Focus = Communication between objects

- Types

- | | |
|-----------------------------------|---------------------------|
| - Observer Pattern | - Mediator pattern |
| - Strategy pattern | - State pattern |
| - Command pattern | - Template Method pattern |
| - Chain of responsibility pattern | - Memento pattern |
| - Iterator pattern | |

- 1) Observer Pattern = (i) One to many dependency; change in one object notifies all dependents
(ii) Used in: event listeners, GUI
- 2) Strategy Pattern = (i) Defines a family of algorithms; objects can choose the algorithm at runtime
(ii) eg - Sorting strategies
- 3) Command Pattern = (i) Encapsulates a request as an object
(ii) Used in: Undo/Redo operations
- 4) Chain of Responsibility Pattern = (i) Passes a request along a chain of handlers
(ii) eg - Technical support escalation

- 5) Iterator Pattern = (i) Provides a way to access elements of a collection without exposing structure
(ii) Mediator controls interactions
- 6) Mediator Pattern = (i) Reduces communication complexity between objects
(ii) Mediator controls interactions
- 7) State Pattern = (i) Allows an object to change behaviour based on its internal state
(ii) eg - vending machine states
- 8) Memento Pattern = (i) Saves an object's internal state for restoring later
(ii) eg - editor undo feature

Advantages of design patterns -

- (i) Reduce development time
- (ii) Provide proven solutions
- (iii) Improve code maintainability
- (iv) Promote loose coupling
- (v) Enable easy testing & debugging

Applications -

- 1) Software architecture
- Used to design layered, modular, scalable systems
- Helps separation of concerns & reduces coupling
- Common patterns: Facade, Singleton, Factory
- 2) Object creation
- Controls object creation to reduce dependency on concrete classes
- Widely used in frameworks & libraries
- Patterns: Singleton, Factory method, abstract factory, Builder
- eg - database connection using singleton
- 3) UI Development
- Patterns: Facade, Command, Strategy, Observer, Composite, Decorator, MVC
- eg - Observer for button click events
- 4) Enterprise applications
- Patterns: Facade, Command, Strategy
- eg - Strategy for payment processing
- 5) Web application development
- Patterns: MVC, Factory, Singleton, Front Controller
- eg - MVC in Spring/Django

- 6) Distributed Systems
 - Manages communication between remote objects
 - Patterns: Proxy, Adapter, Mediator
 - eg- proxy for remote service access
- 7) Game development
 - Patterns: State, Flyweight, Observer
 - eg- Flyweight for repeated game objects
- 8) Real time systems
 - Patterns: Singleton, Observer, Command
 - eg- Command pattern in ~~control~~ control systems
- 9) Mobile applications
 - Patterns: Observer, Strategy, MVC, MVVM
 - eg- Observer for push notifications
- 10) Software maintenance & Reusability
 - Patterns: Decorator, template
- 11) Testing & debugging
 - Patterns: Factory, Dependency injection
- 12) Security Systems
 - Patterns: Proxy, Singleton